Big Data (MEI)



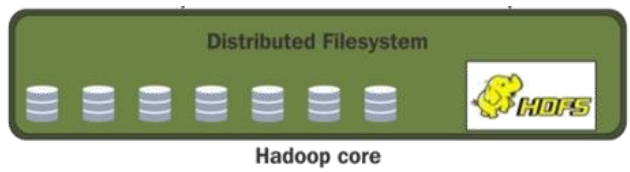
MapReduce

João Pedro Costa
jcosta@isec.pt

1

Distributed File System

- Fault-tolerance?
- Performance?
- Replicate the data?
- Partition the data?
- Consistency?
- Transparency?



2

How to process the data

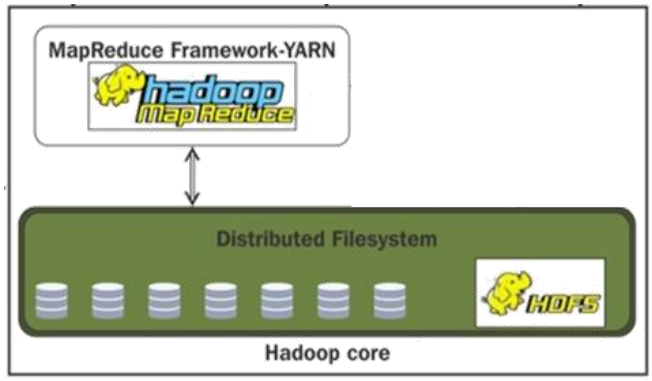
- ???
- Using hundreds of data nodes
- The storage issue is solved

- But how to process the data?
- Network reads of TB or PB of data?

- Processing is now the issue!!!

3

Map Reduce



4

Example Task: Word Count

- We have a huge text document
- Count the number of times each distinct word appears in the file
- **Sample application:**
  - Analyze web server logs to find popular URLs

5

What is the output?

```
echo aa bb aa bb cc aa bb | tr -s ' ' '\n' | sort | uniq -c
```

```
3 aa
3 bb
1 cc
```

```
This can give you a
hint about the base
idea of mapReduce
```


6

What is the output?

```
echo aa bb aa bb cc aa bb | tr -s ' ' '\n' | sort | uniq -c
| awk ' { t = $1; $1 = $2; $2 = t; print; } '
```

```
aa 3
bb 3
cc 1
```

```
This can give you a
hint about the base
idea of mapReduce
```


Task: Word Count

case 1:

- file too large for memory, but all *<word, count>* pairs fit in memory

case 2:

- count occurrences of words:
  `words(doc.txt) | sort | uniq -c`
  - where **`words`** takes a file and outputs the words in it, one per a line

- case 2 captures the essence of **MapReduce**
  - Great thing is that it is naturally parallelizable

Programming Model: MapReduce

- Huge computation over large volumes of data
- Builds an index (sort) on all the data
- Giant computation on lots of computers

- Need a framework that allows to run a computation over large volumes of data

9

Programming Model: MapReduce

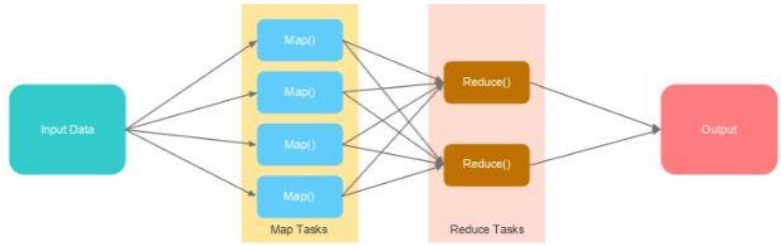
- MapReduce use distributed and parallel algorithms to transform big data sets into a manageable one

- **Map()**
  - performs sorting and filtering of data
  - organizing them in the form of group.
  - generates key-value pairs results, which is later on processed by Reduce

- **Reduce()**
  - does the summarization by aggregating the mapped data
  - takes the output generated by Map as input and combines those tuples into smaller set of tuples

10

Programming Model: MapReduce



11

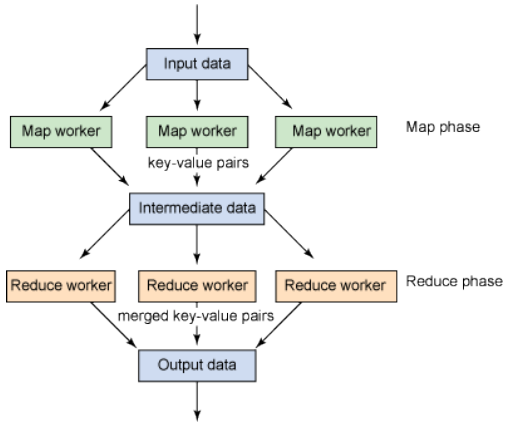
MapReduce: Overview

1. sequentially read a lot of data
2. **`map`**
   - Extract something you care about
3. **`group by key:`** Sort and Shuffle
4. **`reduce`**
   - aggregate, summarize, filter or transform
5. write the result

Outline stays the same, **Map** and **Reduce** change to fit the problem

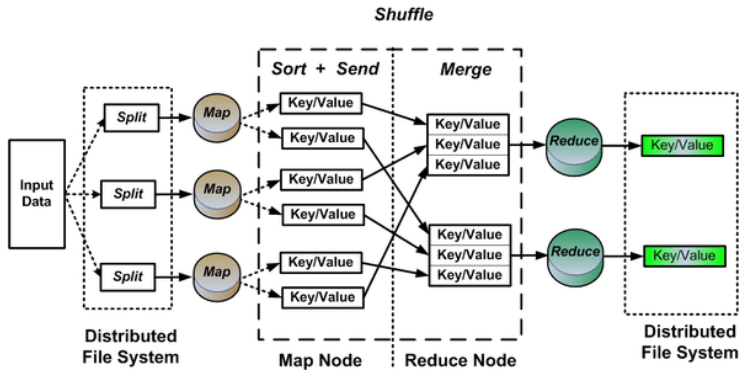
12

MapReduce: Overview



source: http://www.ibm.com/developerworks/java/library/l-hadoop-3/index.html

MapReduce: Overview

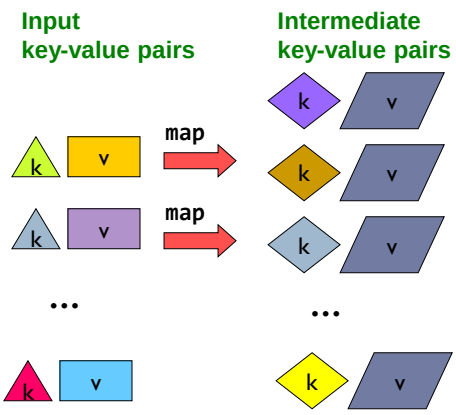


source: https://www.datacamp.com/tutorial/tutorial-cloudera-hadoop-tutorial

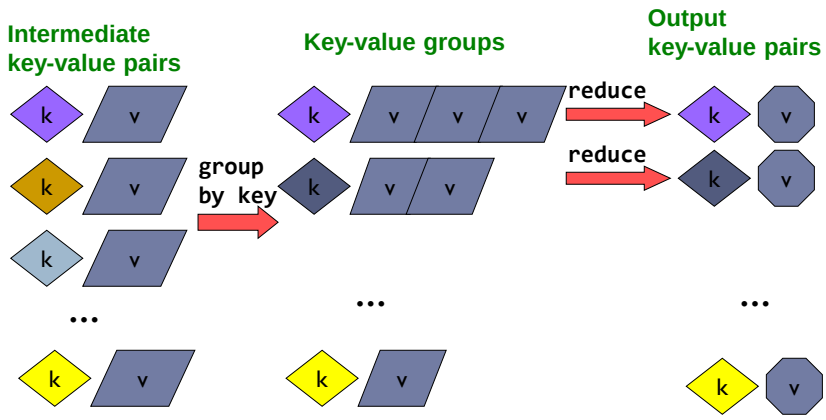
MapReduce: More Specifically

- **input:** a set of key-value pairs
- programmer specifies two methods:
  - `map(k, v)` → `<k', v'>*`
    - takes a key-value pair and outputs a set of key-value pairs
      - E.g., key is the filename, value is a single line in the file
    - there is one `map` call for every `(k,v)` pair
  - `reduce(k', <v'>*)` → `<k', v''>*`
    - all values `v'` with same key `k'` are reduced together and processed in *v'* order
    - there is one `reduce` function call per unique key `k'`

MapReduce: The Map Step

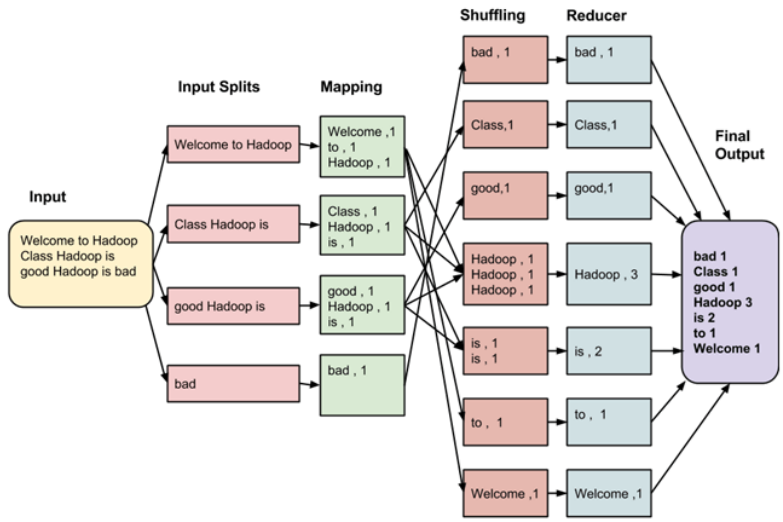


MapReduce: The Reduce Step



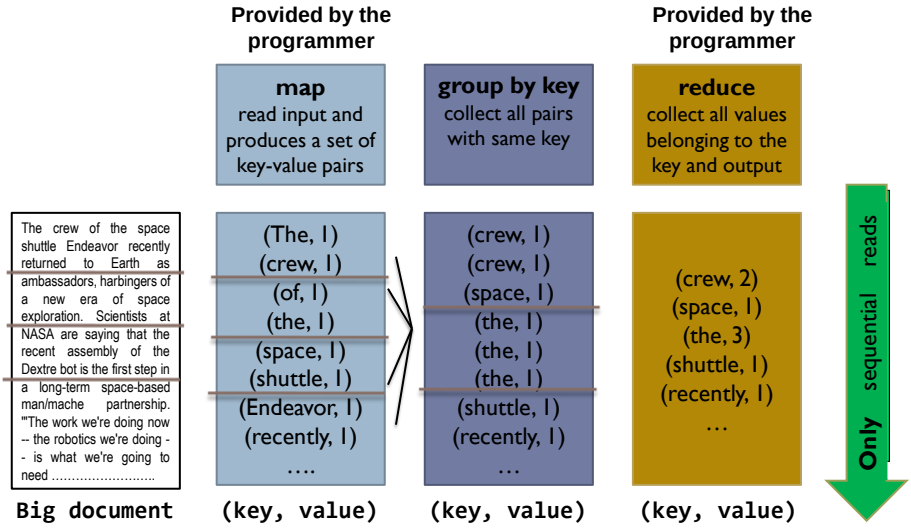
18

MapReduce: Word Count



20

MapReduce: Word Count



MapReduce: Word Count

```
map(key, value):
/* key: document name; value: text of the document */
   for each word w in value:
       emit ( w, 1 )

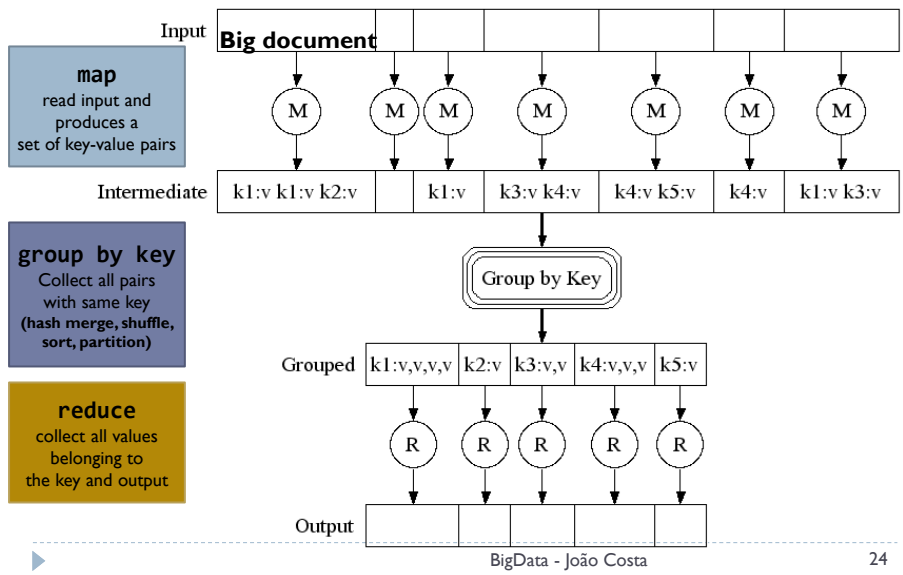

reduce(key, values):
/* key: a word; value: an iterator over counts */
   result = 0
   for each count v in values:
       result += v
   emit ( key, result )
```

MapReduce: Environment

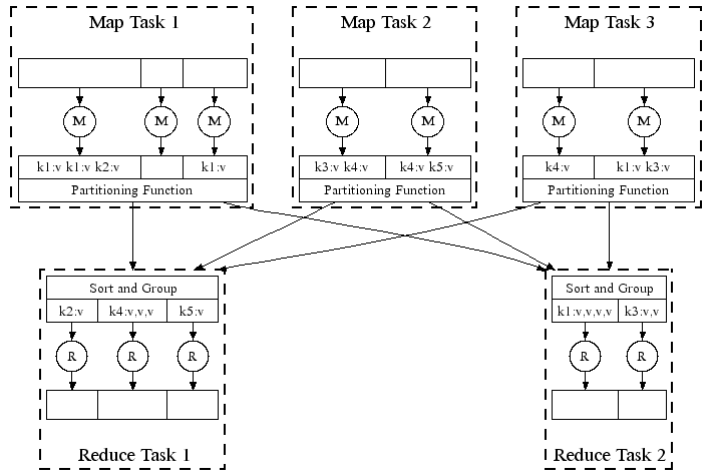
MapReduce environment takes care of

- partitioning the input data
- scheduling the program's execution across a set of machines
- performing the **group by key** step
- handling machine failures
- managing required inter-machine communication

MapReduce: A diagram



MapReduce: In Parallel

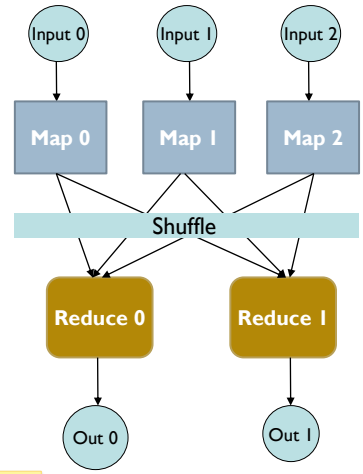


All phases are distributed with many tasks doing the work

25

MapReduce

- Programmer specifies:
  - the **input files**
  - the **map** and **reduce** code
- Workflow:
  - read inputs as a set of key-value-pairs
  - **map** transforms input `kv-pairs` into a new set of `k'v'-pairs`
  - sorts & shuffles the `k'v'-pairs` to output nodes
  - all `k'v'-pairs` with a given `k'` are sent to the same **reduce**
  - **reduce** processes all `k'v'-pairs` grouped by key into new `k''v''-pairs`
  - Write the resulting pairs to files



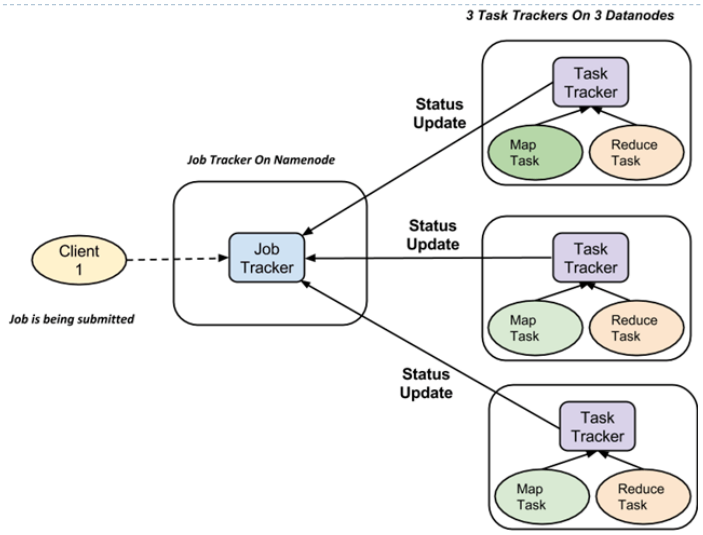
All phases are distributed with many tasks doing the work

26

MapReduce: Data Flow

- **input** and **final output** are stored on a **distributed file system**
  - scheduler tries to schedule map tasks "close" to physical storage location of input data
- **intermediate results are stored on local FS of map and reduce workers**
- **output is often input to another MapReduce task**

Coordination: Master



Coordination: Master

- **Master node takes care of coordination:**
  - **Task status:** idle, in-progress, completed
  - **Idle tasks** get scheduled as workers become available
  - When a map task completes, it sends the master the location and sizes of its *R* intermediate files, one for each reducer
  - Master pushes this info to reducers
- Master pings workers periodically to detect failures

Dealing with Failures

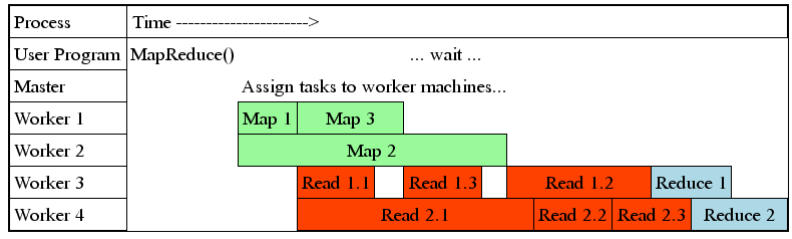
- **map worker failure**
  - **map** tasks completed or in-progress at worker are reset to idle
  - **reduce** workers are notified when task is rescheduled on another worker
- **reduce worker failure**
  - only in-progress tasks are reset to idle
  - reduce task is restarted
- **Master failure**
  - MapReduce task is aborted, and client is notified

How many Map and Reduce tasks?

- ***M*** map tasks, ***R*** reduce tasks
- **rule of a thumb:**
  - make ***M*** much larger than the number of nodes in the cluster
  - one DFS chunk per map is common
  - improves dynamic load balancing and speeds up recovery from worker failures
- **usually, *R* is smaller than *M***
  - because output is spread across *R* files

Task Granularity & Pipelining

- **Fine granularity tasks:** map tasks >> machines
  - Minimizes time for fault recovery
  - Can do pipeline shuffling with map execution
  - Better dynamic load balancing

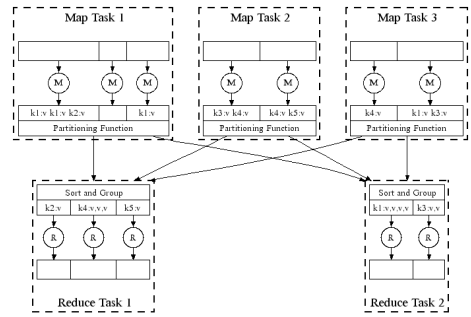


Refinements: Backup Tasks

- **problem**
  - slow workers significantly lengthen the job completion time
    - other jobs on the machine
    - bad disks
    - weird things
- **solution**
  - near end of phase, spawn backup copies of tasks
    - whichever one finishes first "wins"
- **effect**
  - dramatically shortens job completion time

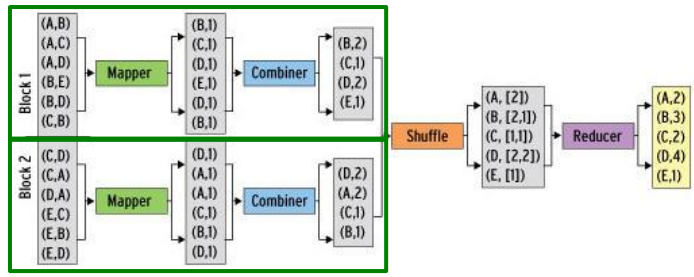
Refinement: Combiners

- often a Map task will produce many pairs of the form $(k,v_1), (k,v_2), \ldots$ for the same key k
  - e.g., popular words in the word count example
- can save network time by **pre-aggregating values** in the mapper**:**
  - combine(k, list(v_1)) → v_2
  - combiner is usually same as the reduce function
- works only if reduce function is commutative and associative



Refinement: Combiners

- **back to our word counting example**
  - combiner combines the values of all keys of a single mapper (single machine)



  - much less data needs to be copied and shuffled!

Refinement: Partition Function

- **to control how keys get partitioned**
  - inputs to map tasks are created by contiguous splits of input file
  - reduce needs to ensure that records with the same intermediate key end up at the same worker
- **system uses a default partition function:**
  - `hash(key) mod R`
- **sometimes useful to override the hash function:**
  - e.g. to ensure URLs from a host end up in the same output file
    `hash(hostname(URL)) mod R`

Map Reduce Java Classes

- `class Mapper`

```
/* Maps input key/value pairs to a set of intermediate
   key/value pairs.
   Maps are the individual tasks which transform input
   records into an intermediate record */
```

- `class Reducer`

```
/* Reduces a set of intermediate values which share
   a key to a smaller set of values. */
```


WordCount example: Mapper

```
import java.io.IOException;
// import mapReduce Mapper Class
import org.apache.hadoop.mapreduce.Mapper;
// import io Class
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import java.util.StringTokenizer;

public class TokenizerMapper
       extends Mapper < Object, Text, Text, IntWritable > {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

//  implement the map function
     public void map(Object key, Text value, Context context)
                                    throws IOException, InterruptedException {
    // for each key, process a list of Iterable values
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
         word.set ( itr.nextToken() );
         // write Key,Value pair
         context.write( word, one );
      }
  }
}
```

WordCount example: Reducer

```
import java.io.IOException;
// import mapReduce Mapper and Reducer Classes
import org.apache.hadoop.mapreduce.Reducer;
// import io Class
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

public class IntSumReducer
                  extends Reducer< Text,IntWritable,Text,IntWritable > {
    private IntWritable result = new IntWritable();
    // for each key, process a list of Iterable values
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
           throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      // write Key,Value pair
      context.write(key, result);
    }
}
```


WordCount example: Mapper

```
import java.io.IOException;
// import mapReduce Mapper and Reducer Classes
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

// import io Class
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.util.StringTokenizer;

public class WordCount {
// ...
}
```

WordCount example: Mapper

```
// ...
  public static class TokenizerMapper
       extends Mapper < Object, Text, Text, IntWritable > {

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context)
                  throws IOException, InterruptedException {

      StringTokenizer itr = new StringTokenizer(value.toString());

      while (itr.hasMoreTokens()) {
         word.set ( itr.nextToken() );
         context.write( word, one );
      }
    }
  }
// ...
```


41

WordCount example: Reducer

```
// ...
  public static class IntSumReducer
       extends Reducer< Text,IntWritable,Text,IntWritable > {

    private IntWritable result = new IntWritable();


    public void reduce(Text key, Iterable<IntWritable> values, Context context )
                  throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }
// ...
```


42

WordCount example: Main

```
public class WordCount {
// public static class TokenizerMapper ...
// public static class IntSumReducer   ...

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");

    job.setJarByClass   ( WordCount.class );
    job.setMapperClass  ( TokenizerMapper.class );
    job.setCombinerClass( IntSumReducer.class );
    job.setReducerClass ( IntSumReducer.class );

    job.setOutputKeyClass  ( Text.class );
    job.setOutputValueClass( IntWritable.class );

    FileInputFormat.addInputPath  ( job, new Path(args[0]) );
    FileOutputFormat.setOutputPath( job, new Path(args[1]) );

    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```


WordCount example: Compile & Run

```
#compile
$ export HADOOP_CLASSPATH=$(hadoop classpath)
$ javac -classpath  ${HADOOP_CLASSPATH}:. WordCount.java

# create a jar file
$ jar -cvf WordCount.jar *.class

# run the jar file
$ yarn jar WordCount.jar WordCount /examples/02-mapreduce/input/ \
                  /examples/02-mapreduce/wordcount/output
```

WordCount example: Compile & Run

```
# before run the jar file
# put the data into the input folder
$ hdfs dfs -cp /datasets/books/shakespeare.txt /examples/02-mapreduce/input


# the output folder should not exist. If exists, delete it first
$  hdfs dfs -rm -f -R /examples/02-mapreduce/wordcount/output

# run the jar file
 yarn jar WordCount.jar WordCount /examples/02-mapreduce/input/ \
                  /examples/02-mapreduce/wordcount/output
```


WordCount example: Compile & Run

```
#Makefile
compile:
        javac -classpath  ${HADOOP_CLASSPATH}:. WordCount.java
        jar -cvf WordCount.jar *.class
        rm *.class

run:
        hdfs dfs -rm -f -R /examples/02-mapreduce/wordcount/output
        yarn jar WordCount.jar WordCount /examples/02-mapreduce/input/ \
                  /examples/02-mapreduce/wordcount/output
        hdfs dfs -ls /examples/02-mapreduce/wordcount/output

setup:
        export HADOOP_CLASSPATH=$(hadoop classpath)
        hdfs dfs -mkdir -p /examples/02-mapreduce/input
        hdfs dfs -cp /datasets/books/shakespeare.txt /examples/02-mapreduce/input
```

WordCount example: Run

```
hadoop@10.204.131.26:~/test/wordcount$ hadoop jar WordCount.jar WordCount /input /output
2022-04-26 19:28:59,155 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at
/10.204.131.27:8032
2022-04-26 19:28:59,361 INFO client.AHSProxy: Connecting to Application History server at /10.204.131.26:10200
2022-04-26 19:29:00,048 INFO input.FileInputFormat: Total input files to process : 7
2022-04-26 19:29:00,618 INFO mapreduce.JobSubmitter: number of splits:14
2022-04-26 19:29:00,849 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1650666298829_0002
2022-04-26 19:29:01,221 INFO impl.YarnClientImpl: Submitted application application_1650666298829_0002
2022-04-26 19:29:01,275 INFO mapreduce.Job: The url to track the job:
http://10.204.131.27:8088/proxy/application_1650666298829_0002/
2022-04-26 19:29:01,275 INFO mapreduce.Job: Running job: job_1650666298829_0002
2022-04-26 19:29:39,114 INFO mapreduce.Job: Job job_1650666298829_0002 running in uber mode : false
2022-04-26 19:29:39,116 INFO mapreduce.Job:  map 0% reduce 0%
2022-04-26 19:29:55,584 INFO mapreduce.Job:  map 14% reduce 0%
2022-04-26 19:29:56,599 INFO mapreduce.Job:  map 21% reduce 0%
2022-04-26 19:30:04,753 INFO mapreduce.Job:  map 31% reduce 0%
2022-04-26 19:30:07,804 INFO mapreduce.Job:  map 36% reduce 0%
2022-04-26 19:30:14,927 INFO mapreduce.Job:  map 36% reduce 12%
2022-04-26 19:30:24,083 INFO mapreduce.Job:  map 43% reduce 12%
2022-04-26 19:30:27,123 INFO mapreduce.Job:  map 43% reduce 14%
2022-04-26 19:30:28,131 INFO mapreduce.Job:  map 50% reduce 14%
2022-04-26 19:30:31,185 INFO mapreduce.Job:  map 56% reduce 14%
2022-04-26 19:30:37,290 INFO mapreduce.Job:  map 59% reduce 14%
2022-04-26 19:30:39,313 INFO mapreduce.Job:  map 59% reduce 17%
2022-04-26 19:30:41,329 INFO mapreduce.Job:  map 62% reduce 17%
2022-04-26 19:30:43,356 INFO mapreduce.Job:  map 64% reduce 17%
2022-04-26 19:30:47,407 INFO mapreduce.Job:  map 71% reduce 17%
2022-04-26 19:30:50,448 INFO mapreduce.Job:  map 71% reduce 21%
```


47

WordCount example: Main

```
2022-04-26 19:31:02,581 INFO mapreduce.Job:  map 71% reduce 24%
2022-04-26 19:31:12,678 INFO mapreduce.Job:  map 79% reduce 24%
2022-04-26 19:31:26,912 INFO mapreduce.Job:  map 79% reduce 26%
2022-04-26 19:31:27,922 INFO mapreduce.Job:  map 86% reduce 26%
2022-04-26 19:31:38,030 INFO mapreduce.Job:  map 87% reduce 26%
2022-04-26 19:31:39,048 INFO mapreduce.Job:  map 91% reduce 26%
2022-04-26 19:31:45,107 INFO mapreduce.Job:  map 94% reduce 29%
2022-04-26 19:31:51,186 INFO mapreduce.Job:  map 95% reduce 29%
2022-04-26 19:31:53,203 INFO mapreduce.Job:  map 98% reduce 29%
2022-04-26 19:31:57,246 INFO mapreduce.Job:  map 99% reduce 29%
2022-04-26 19:32:03,306 INFO mapreduce.Job:  map 100% reduce 29%
2022-04-26 19:32:09,359 INFO mapreduce.Job:  map 100% reduce 31%
2022-04-26 19:32:15,404 INFO mapreduce.Job:  map 100% reduce 67%
2022-04-26 19:32:21,457 INFO mapreduce.Job:  map 100% reduce 70%
2022-04-26 19:32:28,518 INFO mapreduce.Job:  map 100% reduce 73%
2022-04-26 19:32:34,577 INFO mapreduce.Job:  map 100% reduce 75%
2022-04-26 19:32:39,627 INFO mapreduce.Job:  map 100% reduce 78%
2022-04-26 19:32:45,681 INFO mapreduce.Job:  map 100% reduce 80%
2022-04-26 19:32:51,965 INFO mapreduce.Job:  map 100% reduce 82%
2022-04-26 19:32:58,063 INFO mapreduce.Job:  map 100% reduce 84%
2022-04-26 19:33:04,121 INFO mapreduce.Job:  map 100% reduce 86%
2022-04-26 19:33:10,163 INFO mapreduce.Job:  map 100% reduce 88%
2022-04-26 19:33:16,212 INFO mapreduce.Job:  map 100% reduce 91%
2022-04-26 19:33:22,263 INFO mapreduce.Job:  map 100% reduce 93%
2022-04-26 19:33:28,309 INFO mapreduce.Job:  map 100% reduce 95%
2022-04-26 19:33:34,363 INFO mapreduce.Job:  map 100% reduce 98%
2022-04-26 19:33:40,410 INFO mapreduce.Job:  map 100% reduce 100%
2022-04-26 19:33:41,428 INFO mapreduce.Job: Job job_1650666298829_0002 completed successfully
2022-04-26 19:33:41,628 INFO mapreduce.Job: Counters: 56
```


48

WordCount example: Main

```
File System Counters
                FILE: Number of bytes read   =1279189875
                FILE: Number of bytes written=1929091374
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read   =553991001
                HDFS: Number of bytes written=565223457
                HDFS: Number of read operations=47
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0

Job Counters
                Killed map tasks     = 8
                Launched map tasks   =21
                Launched reduce tasks= 1
                Data-local map tasks =10
                Rack-local map tasks =11
                Total time spent by all maps in occupied slots    (ms)=892305
                Total time spent by all reduces in occupied slots (ms)=222237
                Total time spent by all map tasks                 (ms)=892305
                Total time spent by all reduce tasks              (ms)=222237
                Total vcore-milliseconds taken by all map tasks      =   892305
                Total vcore-milliseconds taken by all reduce tasks   =   222237
                Total megabyte-milliseconds taken by all map tasks   =913720320
                Total megabyte-milliseconds taken by all reduce tasks=227570688
```


WordCount example: Main

```
      Map-Reduce Framework
                Map input records =20228696
                Map output records=21053509
                Map output bytes=617750450
                Map output materialized bytes=645744120
                Input split bytes=1519
                Combine input records =21053509
                Combine output records=20108779
                Reduce input groups = 20107745
                Reduce shuffle bytes=645744120
                Reduce input records= 20108779
                Reduce output records=20107745
                Spilled Records=60217822
                Shuffled Maps =14
                Failed Shuffles=0
                Merged Map outputs=14
                GC time elapsed (ms)=23580
                CPU time spent (ms)=209820
                Physical memory (bytes) snapshot = 6254284800
                Virtual memory (bytes) snapshot  =38931697664
                Total committed heap usage (bytes)=5615648768
                Peak Map Physical memory (bytes)  = 486506496
                Peak Map Virtual memory (bytes)   =2601582592
                Peak Reduce Physical memory (bytes)=758349824
                Peak Reduce Virtual memory (bytes)=2637250560
```

WordCount example: Main

```
Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=553989482
        File Output Format Counters
                Bytes Written=565223457
```


51

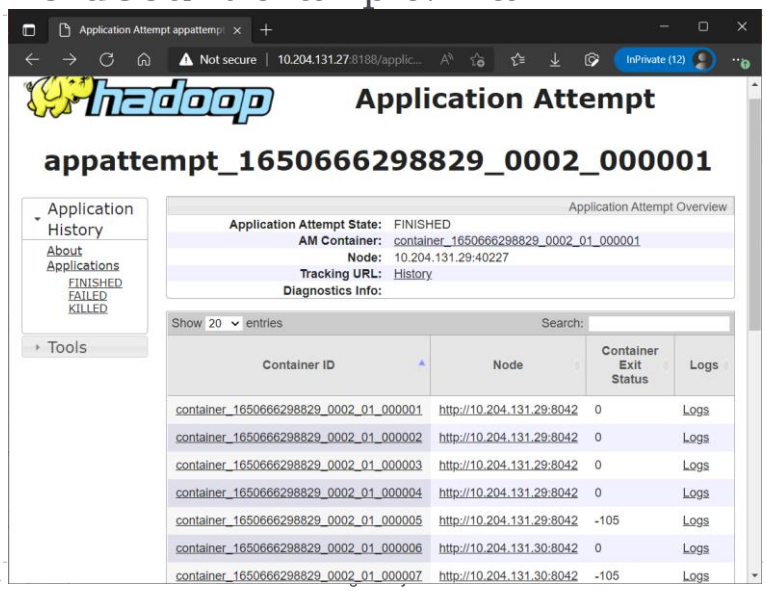
WordCount example: Main

```
$ hdfs dfs -ls /examples/02-mapreduce/wordcount/output/
Found 2 items
-rw-r--r--   2 hadoop supergroup          0 2022-04-26 19:33 /output/_SUCCESS
-rw-r--r--   2 hadoop supergroup  565223457 2022-04-26 19:33 /output/part-r-00000


$ hdfs dfs -cat /examples/02-mapreduce/wordcount/output/part-r-00000 | more
```


52

WordCount example: Main

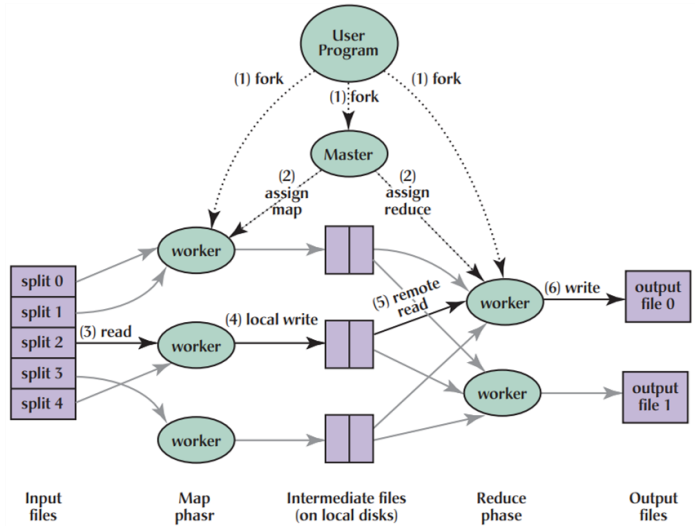


WordCount example

- Contar a frequência das palavras num livro
- Os Lusíadas

http://www.gutenberg.org/cache/epub/3333/pg3333.txt

MapReduce program execution

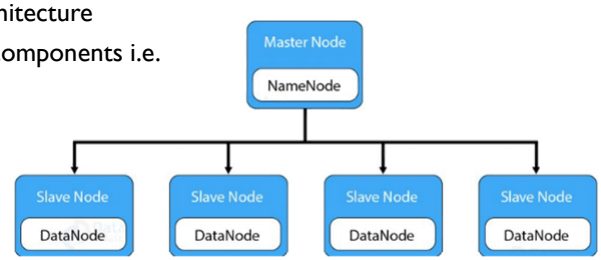


Desafios para a próxima semana

- Escolher um livro
- Contar quantas palavras existem?
- Como apresentar a lista das palavras ordenadas por frequência?

Hadoop HDFS & YARN

- While HDFS is the primary or major component of Hadoop
- responsible for storing large data sets of structured or unstructured data across various nodes
- maintain the metadata in the form of log files.
- has a Master-slave architecture
- consists of two core components i.e.
  - Name node
  - Data Node



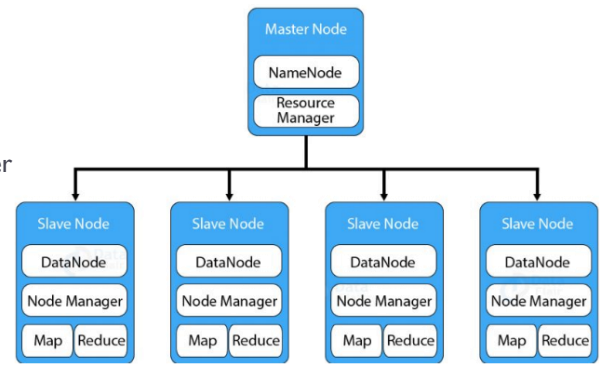
60

Hadoop HDFS & YARN

- Storage nodes usually are also used as processing nodes

- HDFS processes
  - Name node
  - Data Node

- YARN processes
  - Application Manager
  - Resource Manager
  - Node Manager

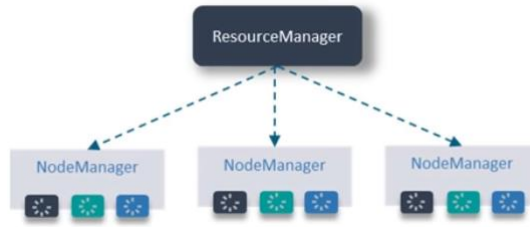


61

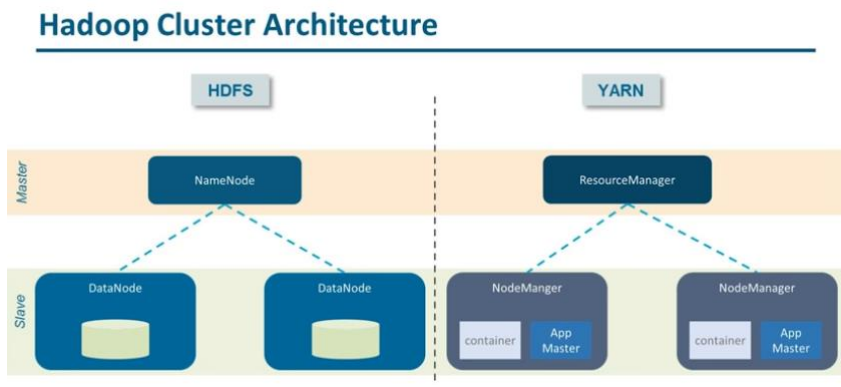
YARN

Yet Another Resource Negotiator

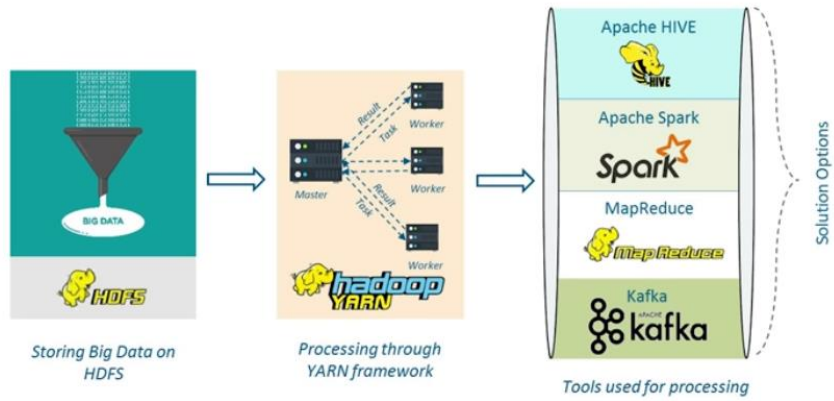
- Provides a paradigm for parallel processing over Hadoop
- Is responsible for integrations of different tools like spark, hive, ..



HDFS & YARN



Hadoop



64

YARN

- is the one who helps to manage the resources across the clusters
- it performs scheduling and resource allocation for the Hadoop System.
- consists of three major components i.e.
  - Resource Manager
  - Node Manager
  - Application Manager
- resource manager allocates resources for the applications in a system
- node managers work on the allocation of resources such as CPU, memory, bandwidth per machine and later on acknowledges the resource manager.
- application manager works as an interface between the resource manager and node manager and performs negotiations as per the requirement of the two

65

YARN

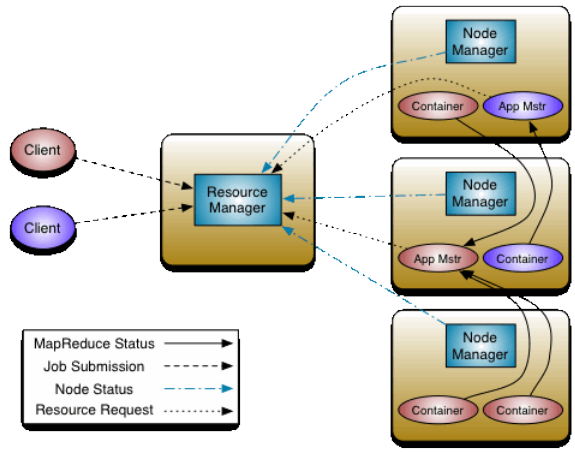
- Resource Manager (RM)
  - receives the processing request
  - allocates resources for the applications in a system
  - passes the requests to corresponding nodemanagers
  - has two main components: Scheduler and ApplicationsManager
- NodeManager (NM)
  - installed on every datanode
  - responsive for execution of task on every datanode
  - allocates resources (CPU, memory, bandwidth, …)
  - later on acknowledges the resource manager
  - is responsible for launching and managing containers on a node
  - containers execute tasks as specified by the AppMaster

RM and NM form the data-computation framework

YARN

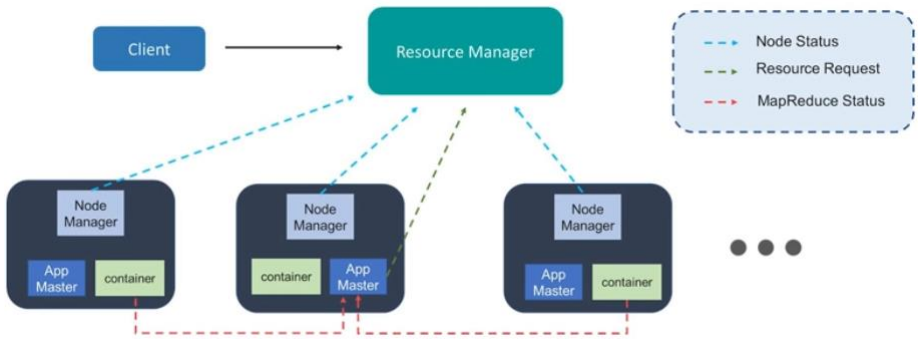
- Application Manager
  - works as an interface between the resource manager and node manager
  - performs negotiations as per the requirement of the two
  - working with the NodeManager(s) to execute and monitor the tasks

YARN Architecture



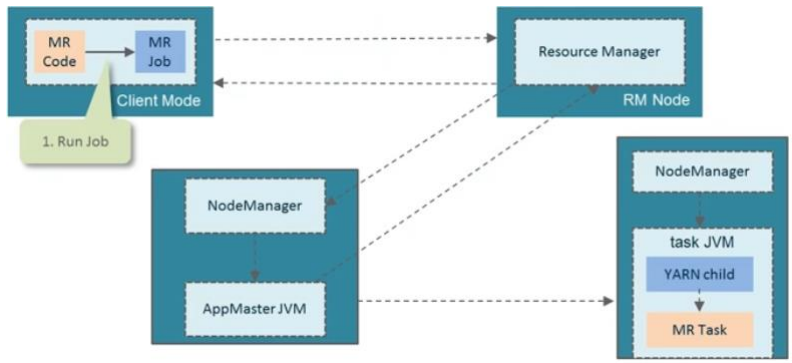
68

YARN Architecture



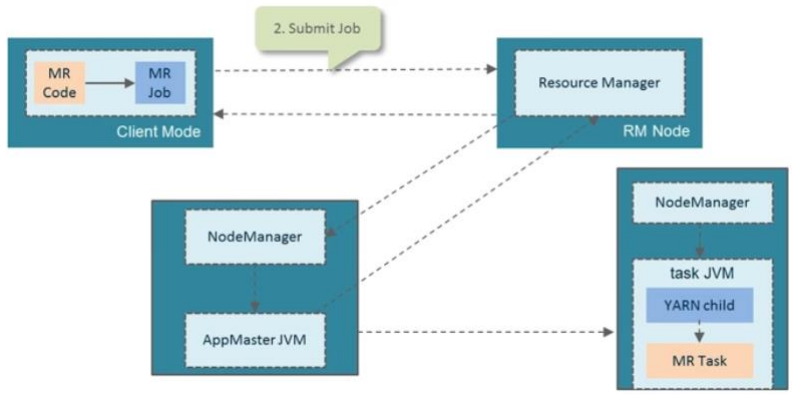
69

YARN- Application Submission



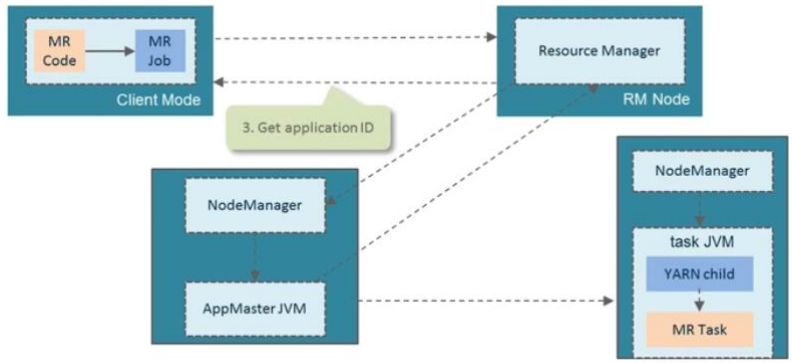
70

YARN- Application Submission



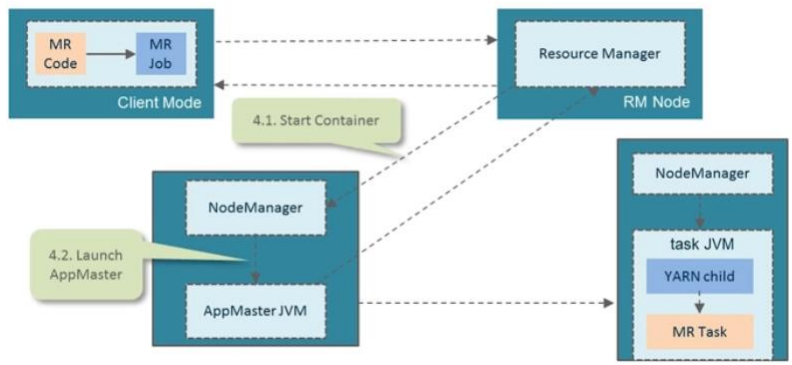
71

YARN- Application Submission



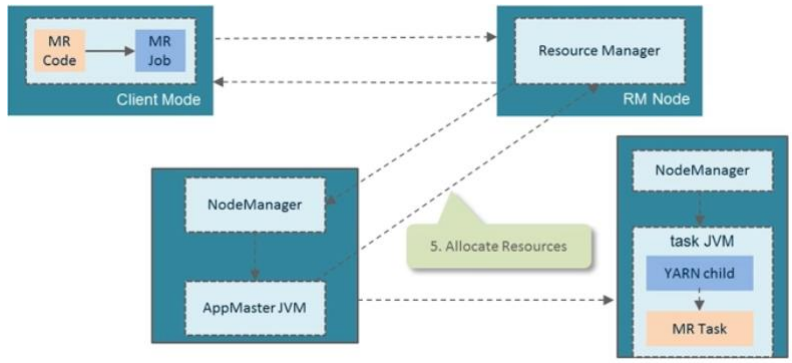
72

YARN- Application Submission



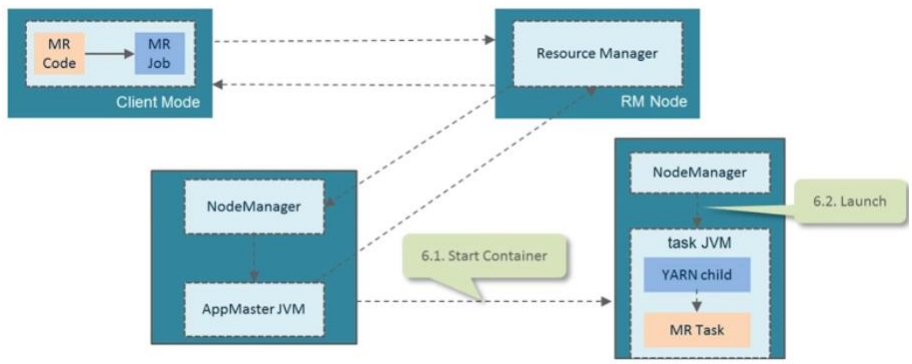
73

YARN- Application Submission



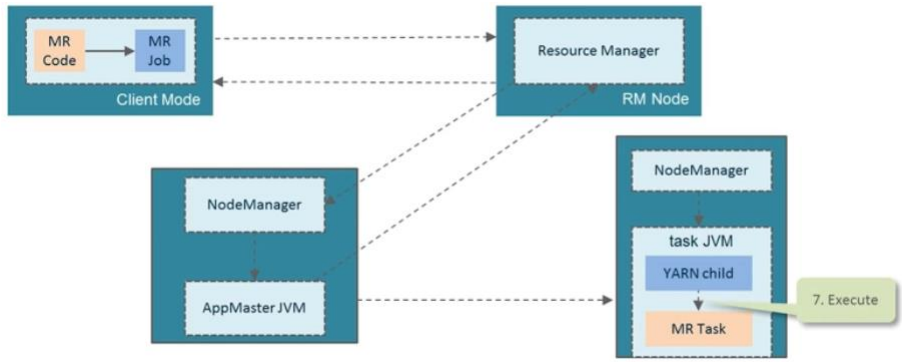
74

YARN- Application Submission



75

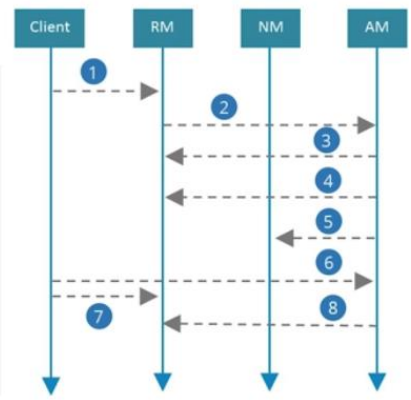
YARN- Application Submission



76

YARN Application workflow

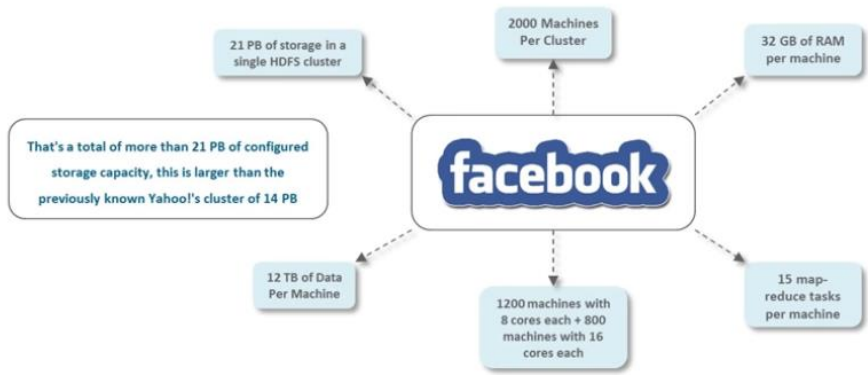
1. Client submits an application
2. RM allocates a container to start AM
3. AM registers with RM
4. AM asks containers from RM
5. AM notifies NM to launch containers
6. Application code is executed in container
7. Client contacts RM/AM to monitor application's status
8. AM unregisters with RM



77

Use Cases

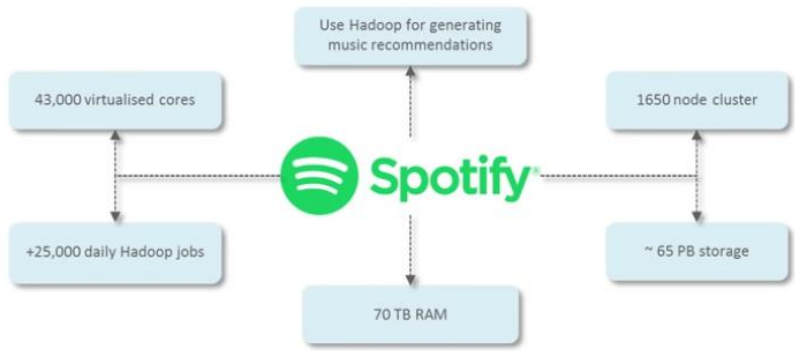
Hadoop Cluster : Facebook Use Case



78

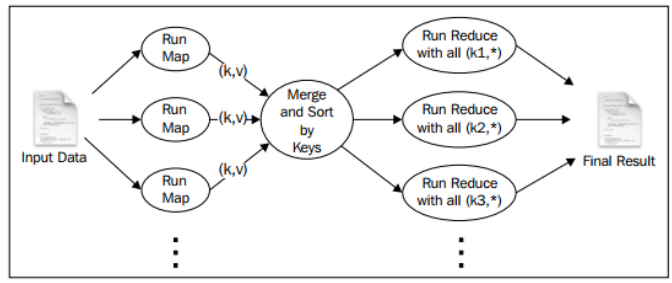
Use Cases

Hadoop Cluster : Spotify Use Case



79

WordCount Sort by Value



80

WordCount Sort by Value

- Sequência de Jobs
- Job1
  - WordCount : `texto` → `K,V`
- Job2
  - Sort By Value : `K,V` → `K,V`

81

Hadoop Streaming

82

Hadoop Streaming

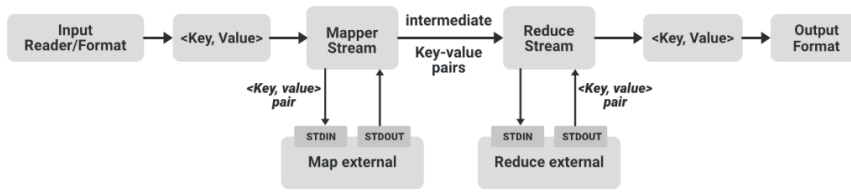
- Hadoop Framework is completely written in java
- but programs for Hadoop are not necessarily need to code in Java programming language.

- Hadoop Streaming (since Hadoop version 0.14.1)
  - allows programmers to write the Map-Reduce programs using different programming languages like Python, Ruby, Perl, C++, etc.
  - any language that can
    - read from the standard input (STDIN)
      - like keyboard input and all
    - write using standard output(STDOUT).

83

Hadoop Streaming

- **How Hadoop Streaming Works**



Example

```
mapred streaming \
    -input /examples/02-mapreduce/input/  \
    -output /examples/02-mapreduce/wordcount/output \
    -mapper /bin/cat \
    -reducer /usr/bin/wc
```

Example

```
mapred streaming \
    -input /examples/02-mapreduce/input/  \
    -output /examples/02-mapreduce/wordcount/output \
    -mapper /bin/cat \
    -reducer /usr/bin/wc
```

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
    -input /examples/02-mapreduce/input/  \
    -output /examples/02-mapreduce/wordcount/output \
    -mapper /bin/cat \
    -reducer /usr/bin/wc
```


Example

```
mapred streaming \
    -input /examples/02-mapreduce/input/  \
    -output /examples/02-mapreduce/wordcount/output \
    -files hdfs://10.84.129.52:9000/datasets/testfile.txt
    -mapper /bin/cat \
    -reducer /usr/bin/wc
```

In each container, create a symbolic link to the file

Python: wordSplitter.py

```
#!/usr/bin/python3
import sys

for line in sys.stdin:
    for word in line.strip().split():
        print(word.lower() + "\t" + "1" )
```

```
echo "foo foo quux labs foo bar quux" | mapper.py
```


91

Python: wordSplitter.py

```
#!/usr/bin/python3
import sys
import re

pattern = re.compile("[a-zA-Z][a-zA-Z0-9]*")
for line in sys.stdin:
    for word in pattern.findall(line):
        print(word.lower() + "\t" + "1" )
```


92

Python: reducer.py

```
#!/usr/bin/python3
import sys

last_key = None
total = 0
for input_line in sys.stdin:

  input_line = input_line.strip()
  this_key, value = input_line.split("\t",)

  value = int (value)

  if last_key == this_key:
     total += value
  else:
     if last_key:
       print("%s\t%s" % (last_key, total) )
     total = value
     last_key = this_key

if last_key:
   print("%s\t%s" % (last_key, total) )
```


94

MapReduce

```
mapred streaming \
                -files  wordSplitter.py,reducer.py \
                -mapper wordSplitter.py \
                -reducer reducer.py \
                -input /datasets/books/ \
                -output /examples/02-mapreduce/wordcount/output
```

-files

Add the files to the container

In each container, create a symbolic link to the files

95